CSCI 5408: Data Management, Warehousing and Analytics

Assignment: Lab - 04

Banner ID: B01026328

Name: Parva Mineshbhai Patel

Date: 2025 – 02 – 02

GitLab Assignment Link: https://git.cs.dal.ca/parva/
CSCI5408_W25_B01026328_ParvaMineshbhai_Patel

Contents

1) Created Tables: User, Wallet, Transaction

DDL Queries

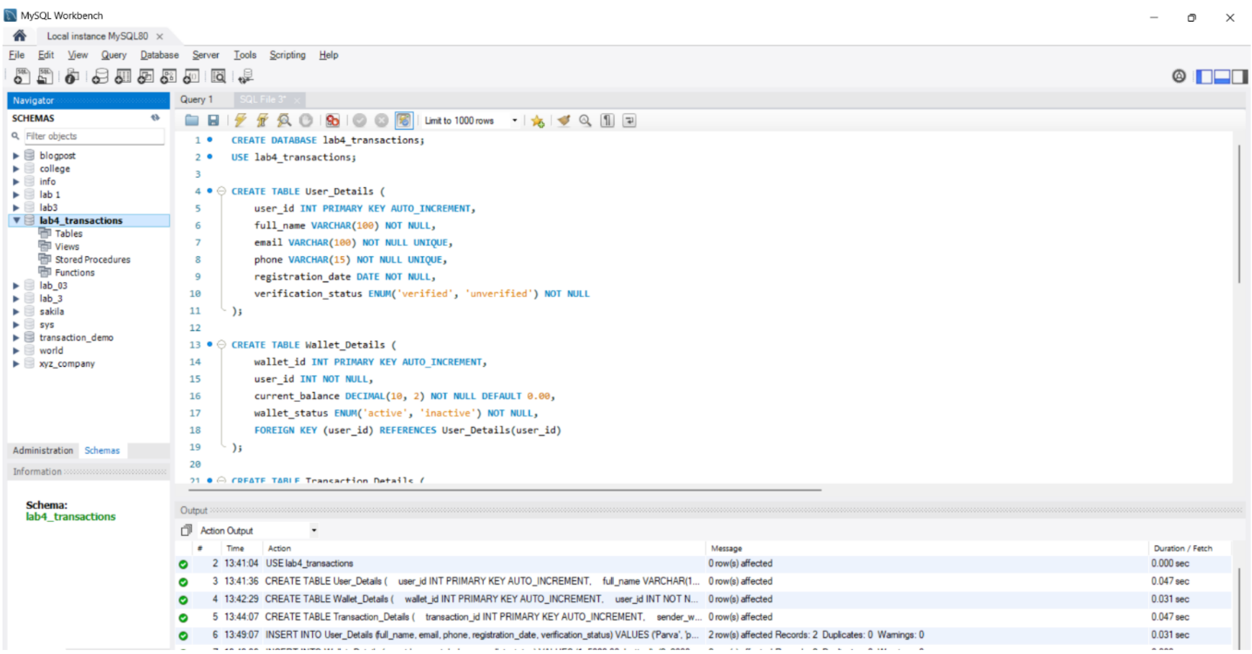
```
CREATE DATABASE lab4_transactions;
USE lab4_transactions;

CREATE TABLE User_Details (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone VARCHAR(15) NOT NULL UNIQUE,
    registration_date DATE NOT NULL,
    verification_status ENUM('verified', 'unverified') NOT NULL
);

CREATE TABLE Wallet_Details (
    wallet_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    current_balance DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
    wallet_status ENUM('active', 'inactive') NOT NULL,
    FOREIGN KEY (user_id) REFERENCES User_Details(user_id)
);

CREATE TABLE Transaction_Details (
    transaction_id INT PRIMARY KEY AUTO_INCREMENT,
    sender_wallet_id INT NOT NULL,
```

```sql
    receiver_wallet_id INT NOT NULL,
    amount DECIMAL(10, 2) NOT NULL,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    transaction_type ENUM('p2p', 'merchant') NOT NULL,
    transaction_status ENUM('pending', 'completed', 'failed') NOT NULL,
    FOREIGN KEY (sender_wallet_id) REFERENCES Wallet_Details(wallet_id),
    FOREIGN KEY (receiver_wallet_id) REFERENCES Wallet_Details(wallet_id)
);
```



Created Tables

2) Inserted Sample Data

DML Queries

INSERT INTO User_Details (full_name, email, phone, registration_date, verification_status)

VALUES

('Parva', 'parva2612@gmail.com', '9028182094', '2025-01-01', 'verified'),

('Dev', 'dev2105@gmail.com', '9452930981', '2025-01-02', 'verified');

INSERT INTO Wallet_Details (user_id, current_balance, wallet_status)

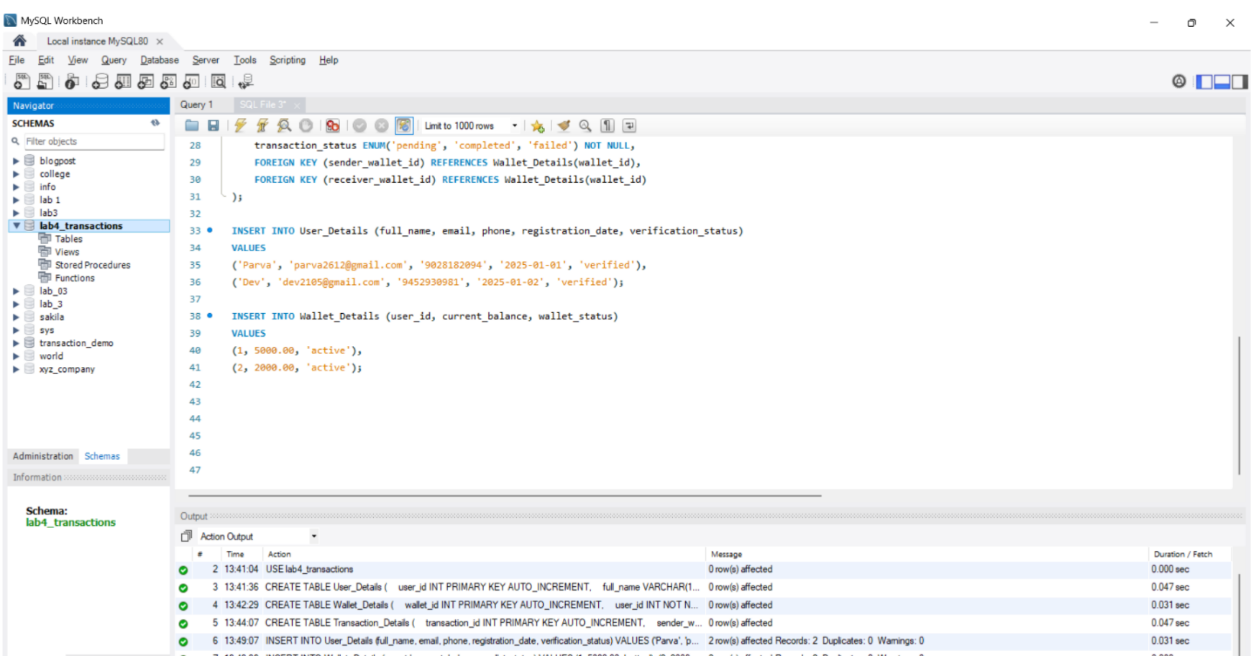
VALUES

(1, 5000.00, 'active'),

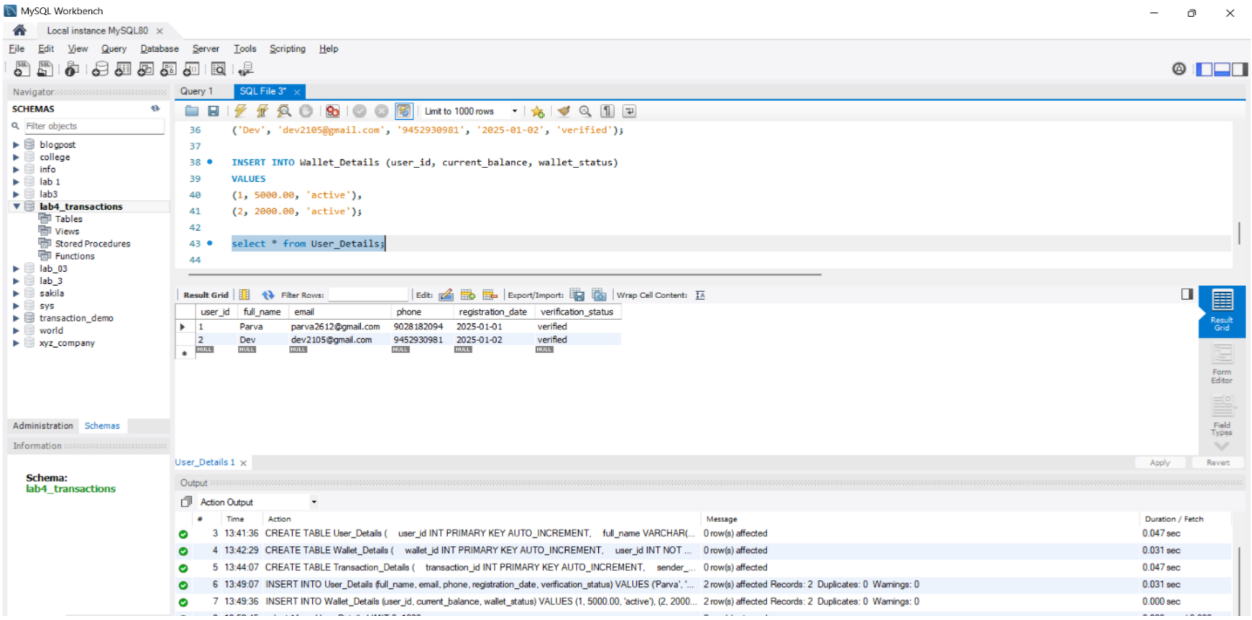
(2, 2000.00, 'active');

select * from User_Details;

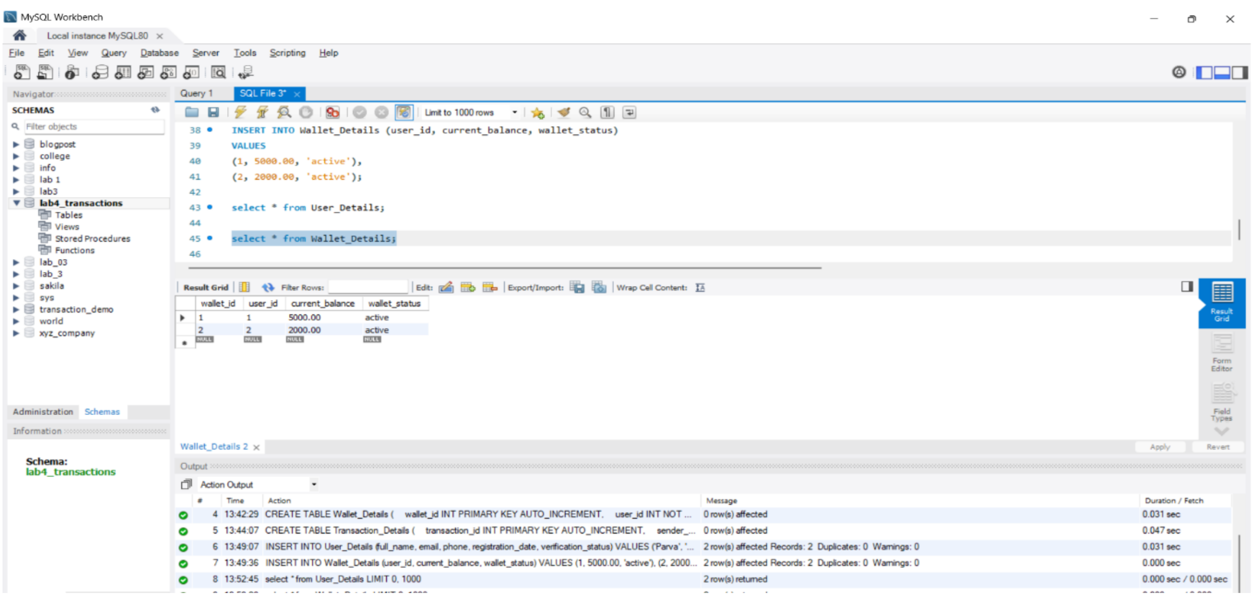
select * from Wallet_Details;



Inserted Sample Data



Display User_Details Table



Display Wallet_Details Table

Explanation

The DDL queries create a database called **lab4_transactions** and created three tables such as **User_Details**, **Wallet_Details**, and **Transaction_Details**, with relationships between them using foreign keys. The **User_Details** table stores user information, the **Wallet_Details** table holds wallet information linked to users, and the **Transaction_Details** table tracks transactions between wallets. The DML queries add sample data for users and their wallets, and the **SELECT** statements are used to display the data that has been inserted.

3) **Transaction Scenario-1: Transaction "Completed" state**

Queries

```
USE lab4_transactions;

SET AUTOCOMMIT = 0;

START TRANSACTION;


UPDATE Wallet_Details

SET current_balance = current_balance - 1000.00

WHERE wallet_id = 1;


SAVEPOINT hold_balance;


INSERT INTO Transaction_Details (sender_wallet_id, receiver_wallet_id, amount,
transaction_type, transaction_status)

VALUES (1, 2, 1000.00, 'p2p', 'pending');


SAVEPOINT transaction_pending;


UPDATE Wallet_Details

SET current_balance = current_balance + 1000.00

WHERE wallet_id = 2;


SAVEPOINT credit_wallet;


UPDATE Transaction_Details

SET transaction_status = 'completed'

WHERE transaction_id = LAST_INSERT_ID();
```

```sql
COMMIT;


SELECT * FROM Wallet_Details;

SELECT * FROM Transaction_Details;
```

Explanation

1. **Start Transaction**: The transaction is initiated with **START TRANSACTION**, meaning the changes made will be grouped together and committed at once.

2. **Deduct Amount from Parva's Wallet**: The balance of Parva's wallet **(wallet_id 1)** is reduced by 1000.00 to hold the amount for the transaction.

3. **Savepoint after Holding Parva's Balance**: A **savepoint** named **hold_balance** is created, marking the point after Parva's wallet balance has been deducted, allowing for a potential rollback.

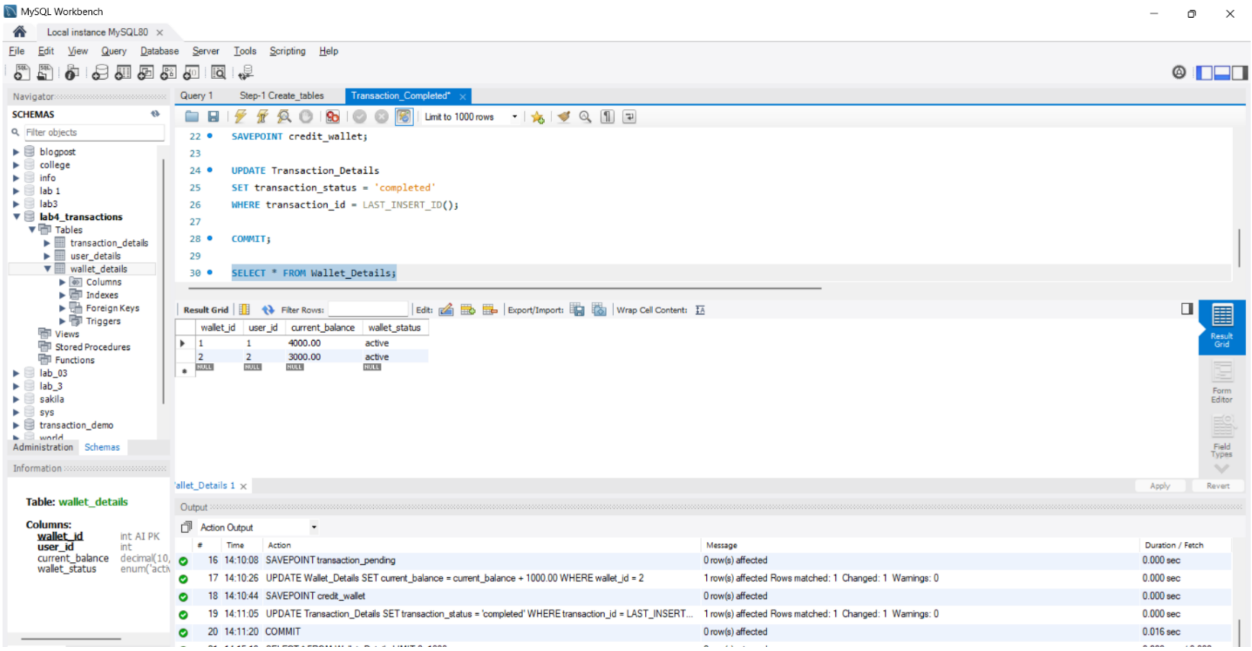
4. **Insert Transaction Record**: A record is inserted into the **Transaction_Details** table with the transaction details, marking the status as pending.

5. **Savepoint after Inserting Transaction Record**: Another **savepoint** named **transaction_pending** is created, allowing for rollback after inserting the transaction record.

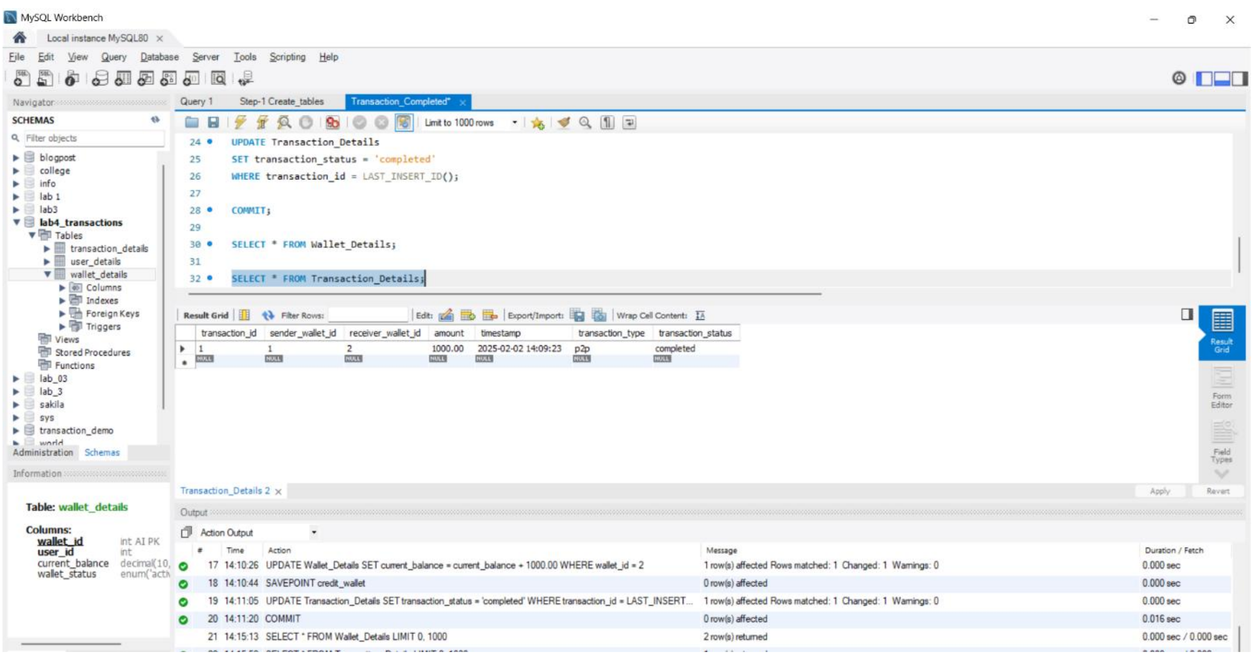
6. **Credit Dev's Wallet**: Dev's wallet **(wallet_id 2)** is credited with 1000.00, completing the transaction.

7. **Savepoint after Credit to Dev's Wallet**: A **savepoint** named **credit_wallet** is created after Dev's wallet is credited, enabling rollback if needed.

8. **Update Transaction Status to Completed**: The transaction status is updated to completed, and all the changes are committed, finalizing the transaction.



Amount Transferred



Transaction Completed

4) **Transaction Scenario-2: Transaction “Failed” state**

Queries

```
USE lab4_transactions;
SET AUTOCOMMIT = 0;
START TRANSACTION;

UPDATE Wallet_Details
SET current_balance = current_balance - 500.00
WHERE wallet_id = 1;

SAVEPOINT hold_balance;

INSERT INTO Transaction_Details (sender_wallet_id, receiver_wallet_id, amount,
transaction_type, transaction_status)
VALUES (1, 2, 500.00, 'p2p', 'pending');

SAVEPOINT transaction_pending;

ROLLBACK TO hold_balance;

UPDATE Transaction_Details
SET transaction_status = 'failed'
WHERE transaction_id = LAST_INSERT_ID();

COMMIT;
SELECT * FROM Wallet_Details;
SELECT * FROM Transaction_Details;
```

Explanation

1. **Start Transaction:** The transaction begins with **START TRANSACTION**, grouping the operations for atomic execution.

2. **Deduct Amount from Parva's Wallet:** Parva's wallet **(wallet_id 1)** balance is reduced by 500.00, temporarily holding the amount for the transaction.

3. **Savepoint after Holding Parva's Balance**: A **savepoint** named **hold_balance** is created, allowing rollback to this point if needed.

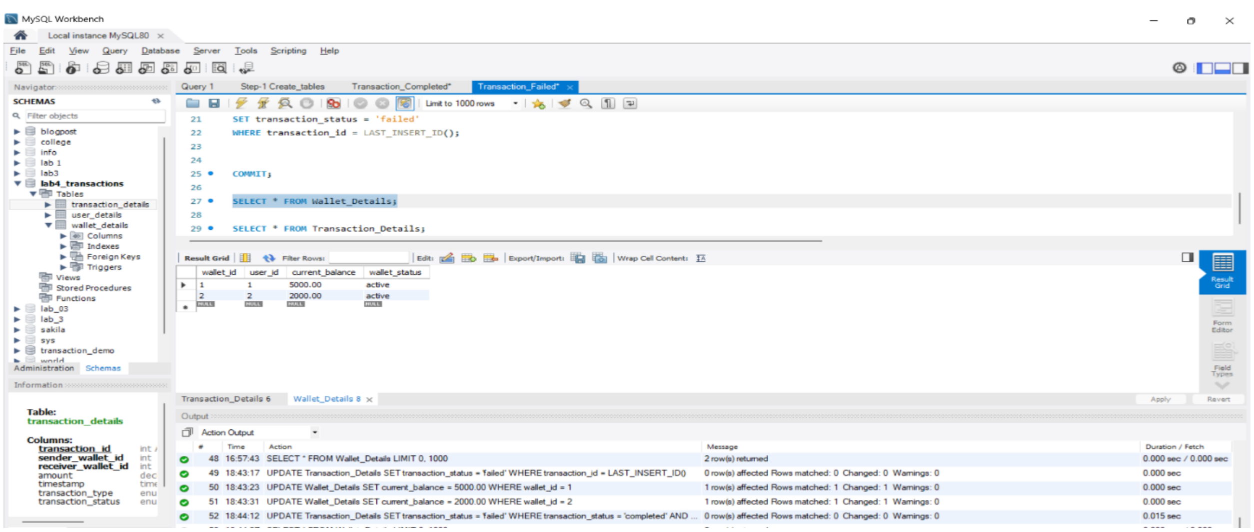
4. **Insert Transaction Record:** A record is inserted into the **Transaction_Details** table, indicating a pending transaction of 500.00 from Parva to Dev**.**

5. **Savepoint after Inserting Transaction Record:** A savepoint named **transaction_pending** is created, marking the point after the transaction record insertion.

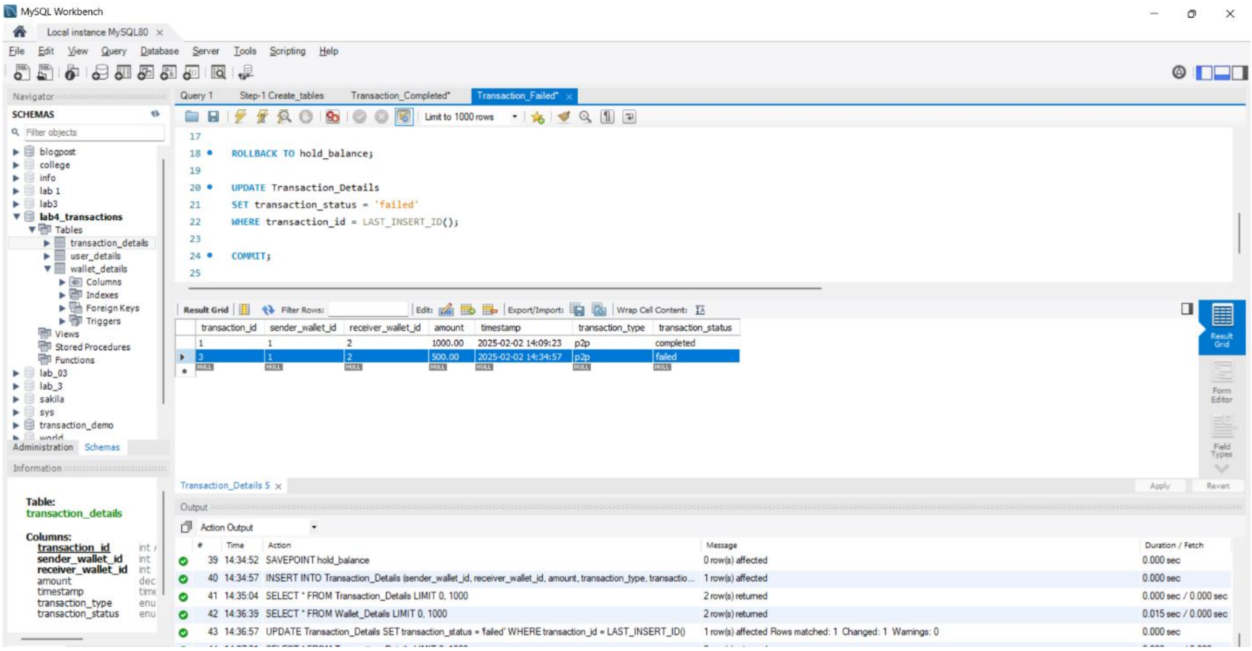
6. **Rollback to Hold Balance Savepoint:** The transaction rolls back to the **hold_balance** savepoint, reversing the deduction from Parva's wallet and canceling the transaction.

7. **Update Transaction Status to Failed:** The transaction status is updated to failed, reflecting that the transaction was not completed.

8. **Commit Changes:** The rollback and status update are committed, finalizing the transaction as failed.



No Transaction in table



Transaction Failed

5) References

1. GeeksforGeeks, “SQL Transactions - COMMIT, ROLLBACK, and SAVEPOINT,” GeeksforGeeks [Online], Available: https://www.geeksforgeeks.org/sql-transactions/ [Accessed: February 02, 2025].

2. Oracle Corporation, "MySQL Workbench - Database Design & Modeling Tool," MySQL [Software], Available: https://www.mysql.com/products/workbench/. [Accessed: February 02, 2025].